

# An End-to-End Learning Approach for Traffic Engineering With Sparse Traffic Measurements

Yingya Guo , Member, IEEE, Zebo Huang, Mingjie Ding, and Huan Luo 

**Abstract**—Centralized Traffic Engineering (TE) plays a critical role in network management, owing to its potential to achieve optimal or near-optimal network performance. However, the overhead from frequent network-wide traffic measurements required to observe the global network view significantly limits its practical use. To address this issue, we propose an end-to-end learning approach called TEST for routing optimization using sparse Traffic Matrices (TMs) obtained from sparse traffic measurements, which require measuring traffic on only a subset of network nodes. Specifically, to mitigate the impact of unknown traffic demands in unmeasured nodes on network performance, we construct a set of synthesized TMs with diverse traffic patterns adaptively to enhance the robustness of the generated routing policies. To address the missed information in sparsely measured TMs, we leverage historical sparse TM sequences to provide sufficient evidence for generating routing policies. To effectively capture the spatio-temporal relationships in the sparse TM sequence, we propose designing a routing model by integrating a Transformer model with a Graph Convolutional Network (GCN). Extensive experiments conducted on topologies with different scales demonstrate that the proposed TEST achieves promising TE performance under sparse traffic measurements. Additionally, discussions on scenarios involving network failures and traffic changes further highlight its robustness.

**Index Terms**—Centralized traffic engineering, sparse traffic matrices, transformer, graph neural network.

## I. INTRODUCTION

**T**RAFFIC Engineering (TE) involves managing traffic distributions by reasonably adjusting routing policies and configuring forwarding devices to avoid network congestion and ensure quality of service (QoS) [1], [2]. Based on pre-computed static routing policies derived from optimization models, traditional TE methods [3], [4], [5], [6], [7] often fail to adapt to dynamic network traffic effectively. Unlike traditional distributed networking architectures, Software-Defined Networking (SDN) [8] offers a more flexible approach by centrally

managing the behaviors of network devices to achieve load balancing in dynamic traffic scenarios. Based on SDN, centralized TE leverages a central entity (e.g., controller) to generate routing policies for geographically distributed forwarding devices, thereby centrally and flexibly manipulating the traffic distributions across the entire network. By the integration of SDN in mobile networks, centralized TE controllers can dynamically orchestrate routing rules, taking into account user mobility and traffic patterns [9]. Nowadays, centralized TE has been a research hotspot due to its potential to achieve optimal or near-optimal network performance through routing optimization based on a global network view.

Recent advancements in Machine Learning (ML) techniques offer promising enhancements for centralized TE, especially in dynamic traffic scenarios. These ML-based TE methods can be broadly categorized into two groups: Supervised Learning (SL)-based TE and Reinforcement Learning (RL)-based TE. SL-based TE methods [10], [11], [12], [13] directly train routing agents (routing models) to learn optimal routing policies from labeled data, which consists of optimal routing solutions derived from global network observations. In contrast, RL-based TE methods [14], [15], [16] train routing agents through frequent interactions with real or simulated network environments in a trial-and-error manner. Based on past routing experience, the trained routing agents learn the relationship between the global network observations and routing policies.

However, to maintain high network performance under dynamic traffic, the existing centralized TE methods need to frequently obtain global network observation to calculate and update routing policies [17], [18]. The key component of these global observation is the traffic information, typically represented as Traffic Matrix (TM), where each element indicates the traffic demand volume between an source-destination pair. In practice, measuring the TM completely and efficiently faces two main challenges. First, frequent network-wide measurement of the TM is resource-intensive, particularly in large-scale networks. Second, complex communication scenarios, such as device failures, node mobility, communication disruptions, and network congestion, often result in unavoidable data loss, hindering the rapid and complete collection of TM data. To handle these challenges, sparse traffic measurements [19], [20], [21] focuses on implementing traffic measurement on a small subset of nodes in the network to obtain sparse TMs,<sup>1</sup> significantly reducing resource overhead and improving measurement

Received 25 September 2024; revised 15 September 2025; accepted 30 October 2025. Date of publication 3 November 2025; date of current version 6 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62002064, in part by the Natural Science Foundation of Fujian Province under Grant 2023J01430 and Grant 2024J01286, and in part by the Fujian Provincial Special Fund for Promoting High-Quality Development of Marine and Fishery Industry under Grant FJHYF-ZH-2023-02. Recommended for acceptance by J. Widmer. (Corresponding author: Huan Luo.)

The authors are with the College of Computer and Data Science, Fuzhou University, Fuzhou 350108, China, also with the Fujian Key Laboratory of Network Computing and Intelligent Information Processing, Fuzhou University, Fuzhou 350108, China, and also with the Engineering Research Center of Big Data Intelligence, Ministry of Education, Fuzhou 350108, China (e-mail: hluo@fzu.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3628341

<sup>1</sup>In this paper, we use 'sparse TM' and 'incomplete TM' interchangeably.

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

efficiency. These sparse TMs, used to reflect overall network conditions, are incomplete TMs that represent only a portion of the total traffic demands in the network.

While sparse traffic measurements has achieved great success in numerous domains, such as network performance monitoring [22], security and anomaly detection [23], and anonymous communication networks [24], the measured sparse TMs still pose significant challenges for TE optimization. Specifically, a complete TM is indispensable for creating training data in SL-based TE methods and for establishing an interactive network environment in RL-based TE methods. Moreover, a sparse TM implies a loss of information, making it challenging to accurately reflect the overall traffic demands of the network environment. Therefore, it is unsuitable to directly leverage sparse TMs for traditional SL-based or RL-based TE methods to train routing agents. Due to the advancements in reconstruction and completion technologies, many studies focused on recovering complete TMs by estimating and filling in the missed traffic data [25], [26], [27] and a straightforward idea is to develop a multi-stage TE approach: (1) reconstructing the complete TMs from the sparse TMs, and (2) training the routing model using the reconstructed complete TMs. However, the inevitable discrepancies between the reconstructed TM and the actual TM will lead to the performance degradation of the multi-stage method.

In order to tackle the challenges of applying sparse TMs collected through sparse traffic measurements to TE optimization, we propose synthesizing TMs to train the routing model by filling in the missing traffic data with diverse traffic patterns, which are generated using different percentiles of the value distribution derived from existing traffic matrix entries. In contrast to a single inaccurately recovered TM, a set of synthetic TMs constructed from different traffic patterns offers greater diversity, thereby improving the routing model's robustness to unknown traffic data. Meanwhile, training the routing model to generate routing policies suitable for different traffic patterns also enhances the TE adaptability and robustness in dynamic network traffic scenarios. Furthermore, to provide sufficient evidence for TE routing optimization under sparse TMs, we use the sequence of TMs, which implies temporal and spatial correlations among traffic flows across network nodes, to describe the network state and mitigate the information loss caused by missed traffic data.

In this paper, we propose a novel end-to-end learning approach, Traffic Engineering under Sparse Traffic (TEST), aimed at reducing traffic measurement overhead in routing optimization. Instead of designing a multi-stage approach, TEST directly treats the sparse TMs as inputs and outputs the optimized routing policies. Specifically, to mitigate the impact of missed traffic data in the sparse TMs derived by sparse traffic measurements, TEST trains a routing model using a set of synthetic TMs. These synthetic TMs are generated by filling in the missed traffic data with a range of adaptively generated traffic patterns, enhancing the diversity of the TMs used for training. After the training procedure, the trained routing model can establish the mapping relationship between sequences of sparse TMs and the corresponding routing policies. To effectively capture the spatio-temporal relationships within these sparse TM sequences, we design the routing model by integrating Transformer model with Graph Convolutional

Network (GCN). The Transformer model leverages attention mechanisms to capture temporal dependencies between different TMs, while the GCN represents each TM as graph-structured data, capturing spatial relationships between nodes in the graph. To evaluate the performance of the proposed approach TEST, we conduct extensive experiments on real network topologies and traffic traces. To validate the superior performance of TEST, we conduct extensive experiments on various network topologies. The comparative experiments demonstrate that TEST achieves significantly better TE performance than other TE methods under sparse traffic measurements conditions.

In a nutshell, the main contributions of this paper can be summarized as follows:

- To reduce the overhead caused by frequent network-wide traffic measurements, we propose an end-to-end learning approach, TEST, which performs TE by optimizing routing based on sparse TM collected through sparse measurements. TEST trains a routing model to intelligently learn the mapping between historical sparse TMs and future routing policies.
- To address the missing network information caused by sparse measurements, we propose introducing sparse TM sequences to provide sufficient evidence for generating robust routing policies. To effectively characterize the network information, we further design a novel routing model that seamlessly integrates a Transformer with a GCN to capture the spatio-temporal relationships in the sparse TM sequences.
- To mitigate the impact of unknown traffic demands at unmeasured nodes on network performance, we propose adaptively constructing a set of synthesized TMs with diverse traffic patterns according to the historical measured traffic demands. Optimizing routing policies under these synthesized diverse TMs enhances the robustness of the generated routing policies to these unknown traffic demands.

The rest of the paper is organized as follows. In Section II, we summarize the related work on TE. In Section III, we provide the motivation and our key insights. Section IV provides a notation of TE model under sparse traffic measurements scenario. Section V introduces our proposed end-to-end supervised learning-based TE method TEST to mitigate performance degradation with sparsely measured incomplete TMs. Section VI evaluates the performance of TEST through extensive experiments. Finally, Section VII concludes this paper. Our code and data will be publicly available at <https://github.com/paper-TEST/TEST>.

## II. RELATED WORK

Traditional TE solutions [28], [29], [30], [31] often focus on designing heuristic algorithms to optimize link weight settings or traffic-splitting ratios between source and destination nodes according to the collected complete traffic demands. Compared to traditional TE solutions, ML-based TE solutions have shown great potential in achieving more intelligent routing optimization, particularly when addressing dynamic network demands.

ML-based TE solutions can be categorized into two paradigms: multi-stage TE methods and end-to-end TE methods.

*Multi-stage TE methods:* TM is a crucial source of information for routing decision-making. Many TE methods implement routing optimization in two steps: (1) predicting the future TM, and (2) determining routing policies based on the predicted TM. Liu et al. [32] demonstrated that Recurrent Neural Network (RNN)-based approaches outperform Convolutional Neural Networks (CNN) and Deep Belief Networks (DBN) in terms of both TM prediction accuracy and TE performance. Troia et al. [33] employed a Gated Recurrent Unit (GRU)-based model to enhance TM prediction accuracy, leading to optimized resource allocation in optical backbone networks. Zheng et al. [34] introduced a flow-by-flow prediction model that leveraged solely temporal correlations among traffic demands with Long Short-Term Memory (LSTM), achieving outstanding TE performance. Due to the temporal relationships among TM sequences, these multi-stage TE methods have achieved some success. However, the inevitable discrepancies between predicted TMs and actual TMs significantly impact their performance.

*End-to-end TE methods:* To mitigate the impact of traffic demand completion and prediction errors on TE performance, some studies design end-to-end optimization methods that directly produce current routing policies according to historical TMs. Valadarsky et al. [14] introduced reinforcement learning (RL) for data-driven routing to determine routing policies based on measured complete traffic demands. Guo et al. [16], [35], [36], [37] trained the flow-splitting agent using RL to establish a direct mapping between traffic demands and flow routing policies, aiming to enhance performance in a hybrid SDN. Ye et al. [10], [11] proposed learning routing policies for the future highly dynamic TMs using GNN-based supervised learning to prevent frequent routing updates and routing performance degradation in future traffic demands. Liu et al. [13] utilizes a deep learning strategy to produce TE solutions through fine-grained path sensitivity constraints, which achieves a balance between performance under normal and peak traffic conditions.

Perry et al. [12] solved the Wide Area Network (WAN) TE problem by modeling it as an end-to-end stochastic optimization problem and leveraged Deep Neural Networks (DNNs) to generate routing policies. However, these existing end-to-end TE methods exhibit superior performance under the implicit assumption that the complete TM is available. As described in the preliminary in Section III, the TE performance obtained by optimizing with sparse TMs decreases as sparsity increases. Therefore, this paper primarily focuses on improving TE performance under incomplete traffic demands collected from sparse traffic measurements. Specifically, we propose to generate various synthetic TMs to fill the missing traffic so that the model could make a more robust routing policy under sparse measurements. Meanwhile, we propose a hybrid model combining the Transformer and GCN modules, where the Transformer captures temporal features across time slots, while the GCN explores spatial features within each observed traffic matrix.

*TE under Sparse Measurements:* Most existing ML-based TE methods share a common assumption that traffic data is complete and easily available, while the cost of network

measurements is often ignored. To reduce the overhead of traffic measurements, some pioneers have explored the practice of reconstructing entire network traffic from partial traffic demands. Xie et al. [38] proposes to apply the tensor completion to infer the complete network traffic data from partial measurements and loss. Xie et al. [39] leverages 3D convolution to capture spatio-temporal features in interaction map, which represents the complex feature correlations among origin, destination, and time slot, to recover the missing traffic. Roughan et al. [40] propose leveraging the estimated traffic matrices to minimize network congestion, which provides a possible way to solve TE problem under sparse measurements. To enhance the practicality of sparse measurements in TE problem, a few pioneer studies have focused on reducing the measurement overhead while performing TE. Le et al. [41] proposes a combination of BiLSTM and GCN to learn the spatio-temporal correlations in partial observed data for imputing network traffic, then a prediction model (e.g., LSTM) is trained on the imputed data to forecast future traffic and inform the routing policy calculation. Le et al. [42] uses compressive sensing technology [21] to reconstruct the complete traffic from the partial maximum flow prediction in each routing cycle, and then obtain the routing policy via LS2SR algorithm [43]. However, the performance of these methods largely depends on the accuracy of the reconstructed or predicted missing traffic under sparse measurements. In contrast to these methods, we propose reconstructing a set of TMs with diverse traffic patterns to enhance the robustness of the generated routing policies against unknown traffic demands.

### III. MOTIVATION AND KEY INSIGHTS

In this section, to explain the motivation behind our proposed method, we first describe the necessity of sparse traffic measurements scenarios and present our observation that TE performance degrades when directly optimizing routing policies based on sparsely measured network traffic. We then highlight our key insight that discrepancies between reconstructed and actual TMs significantly impact TE performance. Optimizing routing for diverse traffic demands may offer an effective solution to improve TE performance under sparse traffic measurements.

#### A. Motivation

*The necessity of sparse traffic measurements:* TM represents the actual traffic volumes collected on routers by network measurement tools, such as NetFlow [44] or OpenTM [45]. These tools typically track packets passing through routers and aggregate those with the same attributes (e.g., source-destination addresses) into network flows, which together form the TM. To obtain a complete TM at a given time, each flow must be accurately measured, incurring a measurement overhead of  $O(n^2)$  for a network with  $n$  routers. As the network scale increases, obtaining a complete TM at fine-grained time intervals becomes challenging due to measurement costs and communication overhead. Therefore, instead of collecting complete TMs, it becomes necessary to introduce sparse traffic measurements to gather sparse TMs from the network [22], [25], [26], [38], [39], [46]. In sparse traffic measurements scenarios, network measurement



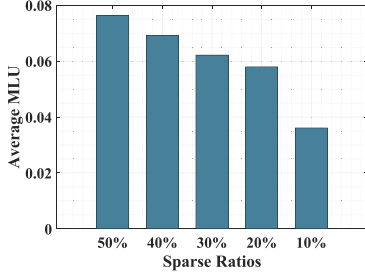


Fig. 1. TE performance degrades by applying the routing policies directly optimized under sparse TMs in Abilene.

tools are deployed only on a subset of routers, reducing the measurement overhead to  $O(nm)$ , where  $m$  is the number of measured routers.

*TE performance degrades by applying the routing policies directly optimized under sparse TMs:* Previous TE studies typically rely on complete TMs as inputs for routing optimization. However, under sparse measurement settings, only sparse TMs are available for generating routing policies, which inevitably degrades network performance. To explain this, instead of using complete TMs, we compute the routing policies using sparse TMs by solving a Multi-Commodity Flow (MCF) problem, as detailed in [10], [47]. Fig. 1 shows the obtained average Maximum Link Utilization (MLU) at varying sparse ratios in the Abilene topology. Here, MLU is a critical metric for measuring network performance in achieving load balancing [14]. The results indicate that as the sparse ratio increases, the average MLU rises, suggesting a degradation in TE performance. Here, the sparse ratio represents the percentage of unmeasured traffic flows relative to the total traffic flows. The primary reason is that, while sparse TMs reduce measurement overhead, the missing traffic information in sparse TMs significantly impacts routing decisions. Therefore, optimizing routing under sparse TMs is both critical and meaningful.

### B. Our Key Insights

*Insight 1. Traffic completion introduces reconstruction errors which impacts the TE performance:* With sparse TMs, a straightforward solution is to first use traffic completion techniques to reconstruct complete TMs. Then, traditional TE methods are applied to optimize routing policies based on the reconstructed TMs. However, inevitable reconstruction errors lead to discrepancies between the actual and reconstructed TMs, which can significantly impact the performance of traditional TE methods. To illustrate this, Fig. 2 provides an example of TE performance of the MCF method under the reconstructed TMs obtained by various traffic completion methods. As shown in Fig. 2(b), based on the measured sparse TM, the reconstructed TMs generated by traffic completion methods, such as CP-ALS [48] and TNCP [49], exhibit slight discrepancies compared to the actual TM. However, when using the reconstructed TMs, the routing policies optimized by the MCF method [16], [47] result in a higher MLU than routing the traffic demands in the actual TM,

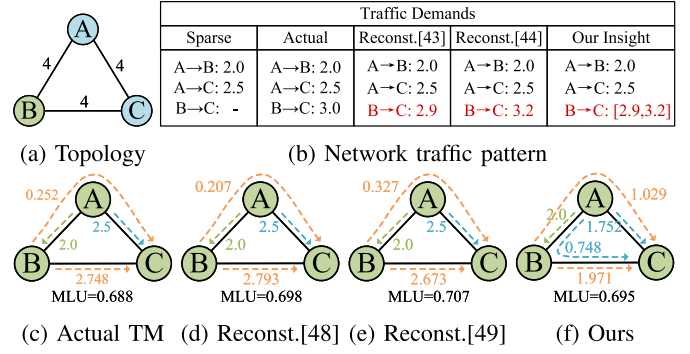


Fig. 2. An example illustrating the impact of reconstruction errors of TM on TE performance: (a) shows the network topology where each link has a capacity of 4, with nodes A and C used to measure traffic demands; (b) presents the actual TM, the measured sparse TM, the reconstructed TMs, and the TMs constructed by our proposed method. (c)–(f) demonstrate the routing policies and actual MLU obtained by the MCF method according to the actual TM, the reconstructed TMs, and our constructed TMs, respectively.

demonstrating that reconstruction errors negatively affect the performance of routing optimization (see Fig. 2(c)–(e)).

*Insight 2. Optimizing routing for diverse traffic patterns offers a promising solution for TE under sparse TMs to mitigate the impact of missed traffic demands:* In sparse measurement scenarios, where measurements are conducted on only a subset of nodes, traffic demands on unmeasured nodes remain unknown. Instead of inaccurately recovering the unobserved traffic demands, we turn to construct a set of TMs by filling unknown traffic demands with diverse traffic patterns. In contrast to a single inaccurately reconstructed TM, a set of TMs constructed from different traffic patterns offers greater diversity, thereby improving the robustness of routing optimization to unknown actual traffic demands. As illustrated in Fig. 2(a), (b), since sparse measurements are limited to nodes B and C, the traffic demand from B to C remains unobserved in the measured sparse TM. By jointly considering traffic patterns 2.9 and 3.2 for the traffic demand from B to C, we can derive the routing policies using the MCF method, as shown in Fig. 2(f). The actual MLU in Fig. 2(f) is lower than Fig. 2(d), (e), indicating that more robust routing policies to the unknown traffic demands are obtained.

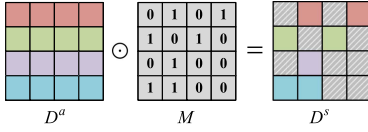
## IV. TE MODEL

In this section, we describe the TE model under sparse traffic measurements. The related notations are summarized in Table I.

- *Network:* The network can be represented as  $G = (V, E)$ , where  $V$  and  $E$  denote the sets of nodes (routers or switches) and links, respectively. Each link  $e \in E$  has a capacity  $c_e \in \mathcal{R}^+$ .
- *Measured Node:* For simplicity, we use a binary mask  $M = [M_v]_{v \in V}$  to represent the nodes where traffic demands are measured. Here,  $M_v = 1$  indicates that node  $v$  is a measured node, and  $M_v = 0$  otherwise.
- *Sparse TM:* The sparse TM is denoted as  $D^s = M \odot D^a$ , where  $\odot$  represents the element-wise product, and  $D^a$  is the actual TM.  $D^s$  is a matrix where each element  $d_{u,v}$

TABLE I  
NOTATIONS

|                         |                                                                                                |
|-------------------------|------------------------------------------------------------------------------------------------|
| $G(V, E)$               | A graph representation of the network topology consisting of a node set $V$ and a link set $E$ |
| $c_e$                   | Capacity of link $e \in E$                                                                     |
| $M$                     | Binary mask to indicate whether the traffic demand is measured                                 |
| $D^a$                   | Actual TM                                                                                      |
| $D^s$                   | Sparse TM                                                                                      |
| $\mathcal{D}_T^s$       | The measured sparse TM sequence at time $T$                                                    |
| $\tilde{\mathcal{S}}_T$ | The set of synthetic TMs for time $T$                                                          |
| $N_L$                   | The number of TMs in sparse TM sequence $\mathcal{D}_T^s$                                      |
| $P_{u,v}$               | The path set for the source-destination node pair $(u, v)$                                     |
| $r_p$                   | The traffic-splitting ratio of traffic flow $d_{u,v}$ transmitted along path $p \in P_{u,v}$   |
| $f_e$                   | Link load on link $e \in E$                                                                    |
| $U_{D^a}^{max}$         | The obtained MLU under TM $D^a$                                                                |

Fig. 3. The illustration of the relationship between sparse TM  $D^s$  and actual TM  $D^a$ .

represents the traffic demand from node  $u$  to node  $v$ . Fig. 3 further illustrates the representation of the sparse TM.

- **Paths:** Each source-destination node pair  $(u, v)$  communicates through at most  $\kappa$  shortest paths, generated by Yen's algorithm [50], and denoted as the set of paths  $P_{u,v}$ .
- **Routing Policies:** The routing policies  $R$  consist of a set of traffic-splitting ratios  $\{r_p\}_{p \in P_{u,v}, u,v \in V}$ , where each ratio  $r_p$  indicates that the traffic flow  $d_{u,v}$  from source  $u$  to destination  $v$  is split and transmitted along path  $p \in P_{u,v}$ . The traffic-splitting ratios  $r_p$  must satisfy the constraint  $\sum_{p \in P_{u,v}} r_p = 1$  to ensure flow conservation.
- **Optimization Goal:** We aim to minimize the Maximum Link Utilization (MLU) to achieve load balancing and avoid network congestion. Given an actual TM  $D^a$  and a routing policies  $R$ , the link load  $f_e$  on link  $e$  is expressed as  $\sum_{u,v \in V, p \in P_{u,v}, e \in p} r_p d_{u,v}$ , and the Maximum Link Utilization (MLU)  $U_{D^a}^{max}$  is calculated as  $\max_{e \in E} (\frac{f_e}{c_e})$ .
- **TE under sparse traffic measurements:** Due to sparse traffic measurements, only sparse TMs  $\{D_t^s\}_{t=0}^{T-1}$ , rather than the actual TMs, can be collected from network until time  $T-1$ . Based on these measured historical sparse TMs  $\{D_t^s\}_{t=0}^{T-1}$ , the TE optimization model deployed on the central controller needs to generate routing policies  $R_T$ , which will be applied for the future TM  $D_T^a$  to minimize MLU  $U_{D_T^a}^{max}$  at time  $T$ .

## V. TEST FRAMEWORK

This section presents our proposed end-to-end learning-based TE approach, TEST, which integrates Transformer model and GCN to optimize routing under sparse traffic measurements.

### Algorithm 1: Adaptive Generation of Synthetic TMs for Training Routing Model.

**Input:**  $\mathcal{D}_T^s = \{D_t^s\}_{t=T-N_L}^T, G$

**Output:**  $\mathcal{D}_{Train} = \{\mathcal{D}_{T-1}^s = \{D_t^s\}_{t=T-N_L}^{T-1}, \tilde{\mathcal{S}}_T\}$

- 1:  $d = \{avg(D_t^s) | D_t^s \in \mathcal{D}_{T-1}^s\} \triangleright$  obtain the average traffic demand for each sparse TM in the TM sequence
- 2:  $\mu_{min} = Q_1(d), \mu_{max} = Q_3(d) \triangleright$  bound  $\mu$  with 25th ( $Q_1$ ) and 75th ( $Q_3$ ) percentiles of  $d$  to remove outliers
- 3: **for**  $\mu = \mu_{min} + \frac{i \times (\mu_{max} - \mu_{min})}{N_\mu}, i \in \{0, 1, \dots, N_\mu\}$  **do**
- 4:   **for**  $D^\delta \in \mathcal{G}(\mu, N_G)$  **do**    $\triangleright$  iterate to generate  $N_G$  samples
- 5:      $\tilde{D}_T = M \odot D_T^s + (1 - M) \odot D^\delta$
- 6:      $\tilde{\mathcal{S}}_T \leftarrow \tilde{\mathcal{S}}_T \cup \tilde{D}_T \triangleright \tilde{\mathcal{S}}_T$  contains a set of synthetic TMs reconstructed from sparse TM  $D_T^s$
- 7:   **end for**
- 8: **end for**

TEST is designed to learn a routing model that establishes the relationship between the routing policies for future TMs and the historical sparse TMs collected through sparse traffic measurements. To train the routing model, a set of TMs is synthesized by filling the missed traffic data with diverse traffic patterns, improving the model's robustness to unknown traffic demands.

This section first describes the adaptive generation of synthetic TMs for training the routing model. Next, we detail the construction of the routing model, which integrates Transformer, temporal attention, and GCN modules. Finally, we explain the training procedure of the routing model. Once the routing model deployed on the central controller is trained, TEST can efficiently generate routing policies based on the collected historical sparse TMs.

#### A. Adaptive Generation of Synthetic TMs

To deal with the unknown traffic demands on unmeasured nodes, we propose to adaptively generate various traffic patterns using statistics of historical traffic demands. These traffic patterns are used to construct a set of synthetic TMs, which serve as training data for the routing model. Specifically, we employ the widely adopted gravity model [51], [52] to simulate the traffic demands on unmeasured nodes. In our work, Modulated Gravity Model (MGM) [53] is used to generate diverse traffic patterns by adaptively adjusting MGM's parameter, i.e., the mean value of traffic volume. This mean value of traffic volume is determined by the statistics of traffic demands in the past TMs.

Algorithm 1 presents the procedure for adaptively generating a set of synthetic TMs for training the routing model. It takes the sparse TM sequence  $\mathcal{D}^s$  and network information  $G$  as inputs, and outputs the training samples  $\mathcal{D}_{Train}$ .  $\mathcal{D}_{Train}$  are constructed by two parts: the sparse TM sequence  $\{D_t^s\}_{t=T-N_L}^{T-1}$  and the synthetic TMs  $\tilde{\mathcal{S}}_T$  for time  $T$ . Specifically, we first compute the average traffic distribution  $d$  from the sparse TM sequence  $\{D_t^s\}_{t=T-N_L}^{T-1}$ . Here, each element in  $d$  represents the mean traffic flow of a TM in  $\tilde{\mathcal{S}}_{T-1}$  (line 1). To generate the possible

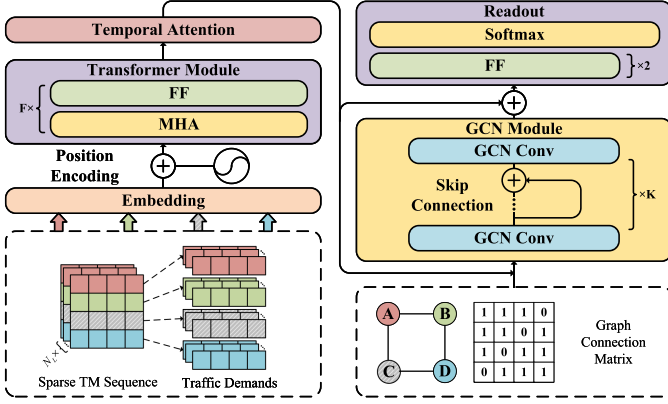


Fig. 4. The routing model used in TEST is mainly composed of five components: the embedding module, transformer module, temporal attention module, GCN module, and readout module. The model takes historical sparse TM sequence and a graph connection matrix as inputs and outputs the routing policies for the future TM.

traffic patterns, the mean traffic volume of MGM is set within  $[\mu_{\min}, \mu_{\max}]$ . To remove the influence brought by outliers, we set  $\mu_{\min}$  and  $\mu_{\max}$  to be  $Q_1(d)$  and  $Q_3(d)$ , respectively (line 2). Subsequently, we gradually increase the mean traffic demand  $\mu$  over the range  $[\mu_{\min}, \mu_{\max}]$  (line 3). Each  $\mu$  is treated as the parameter of MGM  $\mathcal{G}$  to generate  $N_G$  TMs (line 4). For each generated TM  $D^\delta$ , the synthetic TM  $\tilde{D}_T$  at time  $T$  is created by combining the traffic demands of the sparse TM  $D_T^s$  for the measured nodes with the generated TM  $D^\delta$  for the unmeasured nodes (line 5). Finally, we obtain the synthetic TM set  $\tilde{\mathcal{D}}_T$  for time  $T$  by incorporating the synthetic TM  $\tilde{D}_T$  (line 6). The obtained  $\tilde{\mathcal{D}}$  contains a set of synthetic TMs representing various possible traffic.

### B. Routing Model Under Sparse Traffic

To better model the network environment under sparse traffic measurements, the routing model designed in TEST combines the Transformer model with GCN to capture the relationship between the sparse TM sequence and the routing policies. As shown in Fig. 4, the main components of the routing model include embedding, Transformer, temporal attention, GCN, and readout modules. Next, we provide a detailed description of each module in the routing model.

1) *Embedding Module*: To learn a comprehensive representation of sparse TMs, the linear embedding layer takes sparse TM sequence  $\mathcal{D}_{T-1}^s$  as input. For each TM  $D_t^s \in \mathcal{D}_{T-1}^s$ , we obtain the traffic demands  $d_t^v \in \mathbb{R}^{1 \times N}$  collected from each node  $v \in V$  at time  $t$ . Based on  $d_t^v$ , the node feature  $H_{0,t}^v \in \mathbb{R}^{1 \times N_d}$  for node  $v$  at time  $t$  is generated by the embedding layer as follows:

$$H_{0,t}^v = d_t^v W^0 + b^0 \quad (1)$$

where  $W^0 \in \mathbb{R}^{N \times N_d}$  and  $b^0 \in \mathbb{R}^{1 \times N_d}$  are the learnable parameters and  $N_d$  is the feature dimension of  $H_{0,t}^v$ . Note that, to represent the traffic demands on unmeasured nodes, we use a zero vector as a placeholder. Additionally, position encoding based on the time  $t$  when the TM was collected is attached to each node's feature to capture temporal characteristics.

2) *Transformer Module*: To capture the temporal features among sparse TMs, we design a Transformer module consisting of two components: the Multi-Head Attention (MHA) mechanism and Feed-Forward (FF) neural networks. The MHA mechanism enables the model to weigh the importance of traffic feature at different time steps. Compared with other models such as the recurrent neural network (RNN) and long short-term memory (LSTM) models, it can perform parallel operations with greater efficiency. A detailed description of the operation of MHA is given below.

We express the query ( $Q$ ), keys ( $K$ ), and values ( $V$ ) in the MHA according to the input traffic feature as follows:

$$Q = H_1^i W^Q \quad (2)$$

$$K = H_1^i W^K \quad (3)$$

$$V = H_1^i W^V \quad (4)$$

where  $W^Q, W^K \in \mathbb{R}^{N_d \times N_k}$  and  $W^V \in \mathbb{R}^{N_d \times N_v}$  are the learnable parameters in MHA layer. In practice, we set  $N_k$  and  $N_v$  to be  $N_d/h$ , where  $h$  is the number of attention heads. Therefore, the outputs of MHA are shown as follows:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{N_k}}\right)V \quad (5)$$

For each node  $i$ , the traffic feature  $H_1^i \in \mathbb{R}^{L \times N_d}$ , which stacks  $\{H_{0,0}^i, H_{0,1}^i, \dots, H_{0,L-1}^i\}$ , is fed into the transformer module. By passing through  $F$  layers of MHA and FF neural networks, the attention mechanism focuses on the relationships across different time steps of  $H_1^i$ , and finally outputs the temporal traffic features  $H_2^i \in \mathbb{R}^{L \times N_d}$ .

3) *Temporal Attention Module*: The Transformer model extracts traffic features from the TM sequence over  $L$  time steps. To fuse the traffic features from different time steps for routing decisions, we propose a Temporal Attention mechanism that emphasizes the most valuable features by weighing their importance. Therefore, we design the temporal attention module as follows:

$$\alpha = \text{softmax}(H_2^i W^1 + b^1) \quad (6)$$

$$H_3^i = \alpha^T H_2^i \quad (7)$$

where  $W^1 \in \mathbb{R}^{N_d \times 1}$  and  $b^1 \in \mathbb{R}^{L \times 1}$  are the learnable parameters. In (6),  $\alpha \in \mathbb{R}^{L \times 1}$  is the attention coefficient of different time steps normalized by softmax function, which is used to weigh the traffic feature at different time steps. By multiplying  $\alpha^T$  and  $H_2^i$  in (7), a rich information feature for node  $i$  is obtained, denoted as  $H_3^i \in \mathbb{R}^{1 \times N_d}$ .

4) *GCN Module*: GCN, as one of the variants of graph neural networks, offers distinct advantages over traditional neural networks in modeling graph-structured data. The process of multiple message passing in GCN facilitates the integration of features between nodes and the topological structure information into node traffic features. To model the spatial relationship between node traffic feature, we introduce GCN into our TEST model. Due to the simple and efficient computation, GCN is used to learn the spatial features of node based on graph structure. In a GCN, the convolution operation aggregates features from



a node's neighbors to update its traffic feature. As shown in Fig. 4, we design a symmetric  $K$ -layer GCN with an equal number of channels in the symmetric layers. We also apply a skip connection to the  $i$ -th layer and the  $(K - i)$ -th layer of the multi-layer network structure. The propagation rule for GCN can be expressed as follows:

$$H^{(l+1)} = \sigma \left( \bar{D}^{-1/2} \bar{A} \bar{D}^{-1/2} H^{(l)} W^{(l)} \right) \quad (8)$$

$$\bar{A} = A + I \quad (9)$$

where  $H^{(l)}$  is the feature at the  $l$ -th layer,  $W^{(l)}$  is the learnable parameter of the  $l$ -th layer.  $\bar{A}$  is denoted as the connection matrix of the graph with added self-connections, where  $A$  is the original adjacency matrix and  $I$  is the identity matrix.  $\bar{D}$  represents the diagonal node degree matrix of  $\bar{A}$  and  $\sigma$  is the non-linear activation function.

In the first layer of GCN, we stack traffic features for all nodes, denoted as  $H^{(0)} \in \mathbb{R}^{N \times N_d}$ , as one of the GCN module's inputs. To incorporate graph structure information, the graph connection matrix  $\bar{A}$  is also provided as another input. After message passing through the  $K$ -layer GCN, the node traffic feature output by the final GCN layer, denoted as  $H^{(K)} \in \mathbb{R}^{N \times N_d}$ , captures both neighboring node features and topological structure information.

5) *Readout Module*: Two FF layers and a softmax layer comprise the readout module, which predicts the routing policies based on the node traffic features extracted by GCN module. In particular,  $H^{(0)}$  and  $H^{(K)}$  are combined by addition, and the derived traffic features are flattened as inputs for the readout module, denoted as  $H_4 \in \mathbb{R}^{N \times N_d}$ . The routing policies  $R_T$  used for the future TM is generated by the readout module as follows:

$$R_T = \text{softmax}(H_4 W^2 + b^2) \quad (10)$$

where  $W^2 \in \mathbb{R}^{N \times N_d \times O}$  and  $b^2 \in \mathbb{R}^O$  are the learnable parameters.  $O$  equals to  $N \times (N - 1) \times \kappa$ , also indicating the dimension of routing policies  $R_T$ . The routing policies  $R_T$  are represented as a set of traffic split ratios, where each ratio corresponds to the proportion of traffic for a particular source-destination pair that is allocated to each of the  $\kappa$  available paths.

### C. Model Offline Training and Online Execution

Based on the training samples generated by Algorithm 1, TEST trains the routing model offline to establish the relationship between the sparse TM sequence  $\mathcal{D}_{T-1}^s$ , collected up to time  $T-1$ , and the routing policies  $R_T$  that will be applied at time  $T$ . Hence, we denote the output of the routing model as  $R_T = \pi_\theta(\mathcal{D}_{T-1}^s; G)$ . Here,  $\theta$  represents the parameters of the routing model  $\pi$ . Due to the adopted sparse traffic measurements, the complete actual TM  $D_T^a$  cannot be obtained. To predict routing policies that are robust to unknown traffic demands, we aim to determine  $R_T$  by minimizing  $\mathcal{L} = \sum_{\tilde{D}_T \in \tilde{\mathcal{S}}_T} U_{\tilde{D}_T}^{\max}$ . Therefore, the model parameters  $\theta$  can be updated by stochastic

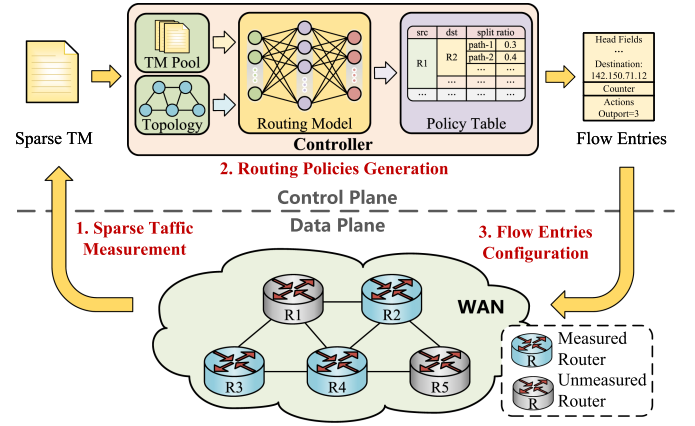


Fig. 5. The illustration of online execution of TEST in WAN.

gradient descent (SGD) [54] as follows:

$$\theta \leftarrow \theta - \beta \cdot \nabla_{\theta} \mathcal{L} \quad (11)$$

where  $\beta$  is the learning rate and  $\nabla_{\theta} \mathcal{L}$  is the gradient of the loss function with respect to the parameters  $\theta$ . In practice, we utilize the ADAM optimizer [55], which combines momentum and adaptive step sizes, to stabilize training and achieve faster convergence.

Once the training procedure is completed, TEST deploys the trained routing model in the central controller, as illustrated in Fig. 5. The entire network can be divided into two parts: the data plane and the control plane. By leveraging the OpenFlow protocol [56], traffic data and control messages can be transmitted between the data plane and the control plane. Specifically, in the data plane, forwarding devices such as switches or routers measure network information, i.e. traffic demands and network topology, using network measurement tools like NetFlow [44] or OpenTM [45]. The measured network information is then periodically reported to the central controller. Due to sparse traffic measurements setup, only a portion of forwarding devices are deployed with network measurement tools, resulting in sparse TMs to be collected and reported to the central controller. In the control plane, the controller receives and stores the sparse TM in a TM pool. Based on the network topology  $G$  and the sparse TM sequence  $\mathcal{D}_{T-1}^s$  extracted from the TM pool, the routing model generates the routing policies  $R_T$  to update the policy table. Finally, the updated policy table is translated into a set of flow entries via the OpenFlow protocol [56]. These flow entries are dispatched to the forwarding devices in the data plane to reconfigure and update their OpenFlow group tables, thereby achieving routing optimization under sparse traffic measurements.

## VI. EVALUATION

### A. Experimental Setup

1) *Datasets*: To evaluate our proposed method TEST, four network topologies of different scales are employed in our evaluation and the detailed information is presented in Table II. As shown in Table II, the first three topologies are America

TABLE II  
THE NETWORK TOPOLOGIES USED IN EVALUATION

| Topology  | Nodes | Edges | Granularity |
|-----------|-------|-------|-------------|
| Abilene   | 12    | 30    | 5 minutes   |
| CERNET    | 14    | 32    | 5 minutes   |
| GEANT     | 23    | 74    | 15 minutes  |
| USCarrier | 158   | 378   | Synthetic   |

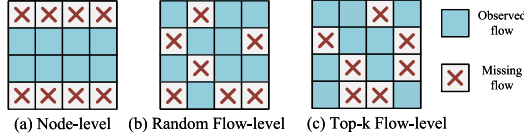


Fig. 6. Illustrations of the sparse TM obtained using different measurement schemes: (a) node-level measurement, (b) flow-level measurement and (c) top-k flow-level measurement.

Research and Education Network (Abilene) with 12 nodes, 30 edges [57], China Research and Education Network (CERNET) with 14 nodes, 32 edges [58], and the Europe Research and Education Network (GEANT) with 23 nodes, 74 edges [59], respectively. The fourth topology USCarrier is the network topology from Topology Zoo [60]. The real network traffic for Abilene, CERNET, and GEANT was measured at granularity of 5 minutes, 5 minutes and 15 minutes, respectively. This corresponds to 288 TMs per day for both Abilene and CERNET, and 96 TMs per day for GEANT. In our experiments, we use TMs with 16 and 8 consecutive days for training and testing on Abilene and CERNET, and TMs with 30 and 15 consecutive days for training and testing on GEANT. For the USCarrier topology, 4,000 synthetic TMs are generated using the gravity model [53] and evenly divided (8:2) into training and testing sets.

To simulate the sparse traffic measurements in all topologies, we generate the sparse TMs by eliminating the traffic data collected on the randomly selected unmeasured nodes. In our experiments, we simulate three different sparse traffic measurement schemes. As shown in Fig. 6, the scenarios considered are: (1) node-level measurement, which measures flows from randomly sampled nodes; (2) flow-level measurement, which samples random origin-destination pairs per time step; and (3) top-k flow-level measurement, which tracks only the  $k$  largest flows. For the top-k scenario, we implement both optimal and practical top-k selection strategies. The optimal top-k strategy samples only the  $k$  largest flows in each TM, whereas the practical top-k strategy [42] maintains stability by retaining 80% of the largest flows and allocating the remaining 20% to random selection for discovering potential new large flows. To eliminate random effects, we repeat the simulations and experiments 1,000 times.

2) *Metric*: In the evaluation, we aim to minimize the MLU to achieve load balancing in the network. Thus, we use the normalized MLU, referred to as the MLU Performance Ratio ( $PR$ ), as the evaluation metric. The definition of  $PR$  is given as

follows:

$$PR = \frac{U}{U_{opt}} \quad (12)$$

where  $U$  represents the MLU obtained by different methods and  $U_{opt}$  represents the optimal MLU obtained by solving the MCF problem [61] through a LP solver according to an actual and complete TM. In the evaluation, a lower  $PR$  value indicates that the routing policies is closer to the optimal routing policies, implying better network performance. Note that, we compute the average  $PR$  value for all the TMs in the testing set as the normalized MLU.

3) *Implementation Details*: All our experiments are conducted on a server with a 36-core Intel 3.0 GHz CPU, 128 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU. We implement the TEST method using PyTorch and utilize Gurobi optimizer as the LP solver [62]. To generate diverse traffic patterns, we empirically set the parameters  $N_G$  and  $N_\mu$  in Algorithm 1 to 24 and 5, respectively. For the routing model, we set sparse TM length  $L$  to 12 and the number of paths  $\kappa$  for each source-destination node pair to 4. Besides, both the initial embedding dimension and the dimensions of all FF sub-layers are set to 128. The number of layers  $F$  in the Transformer module is set to 3, with the MHA layer consists of 8 heads. The number of layers  $K$  used in the GCN module is configured to 6, and the number of channels in the first half of the symmetric GCN is set to  $\{16, 32, 64\}$ , respectively. Additionally, the learning rate  $\beta$  is set to  $10^{-3}$  during the training phase, and the batch size is fixed at 64 for all datasets.

4) *Baselines*: To demonstrate the superiority of our proposed method, we compare TEST with the following baseline TE methods.

- *Omniscient TE* [4]: It models routing optimization as a MCF problem, resulting in optimal network performance. In the experiments, it optimizes routing using complete, actual traffic information, achieving optimal network performance. The MLU obtained is considered as  $U_{opt}$ , which is used to calculate the MLU performance ratio.
- *RACKE* [5], [63]: It calculates a probability distribution over multiple forwarding paths using decomposition trees and directs traffic accordingly, without relying on the actual TMs.
- *Mean Interpolation (MI)*: It first performs mean interpolation based on the graph adjacency structure to complete the sparse traffic matrix. Subsequently, the routing policy is derived through MCF optimization.
- *RL* [14]: It utilizes RL to learn the routing policies from a set of historical TMs. Due to sparse traffic measurements, the historical TMs are constructed by the sparse TMs.
- *NTC* [39]: It is a multi-stage TE scheme that first uses the tensor completion algorithm to reconstruct complete TMs, and then leverages LSTM model to predict the future TM. According to the predicted TM, the routing policy is derived through MCF optimization.
- *MTCR-CS* [42]: It leverages compressive sensing technology [21] to reconstruct the complete traffic from the partial



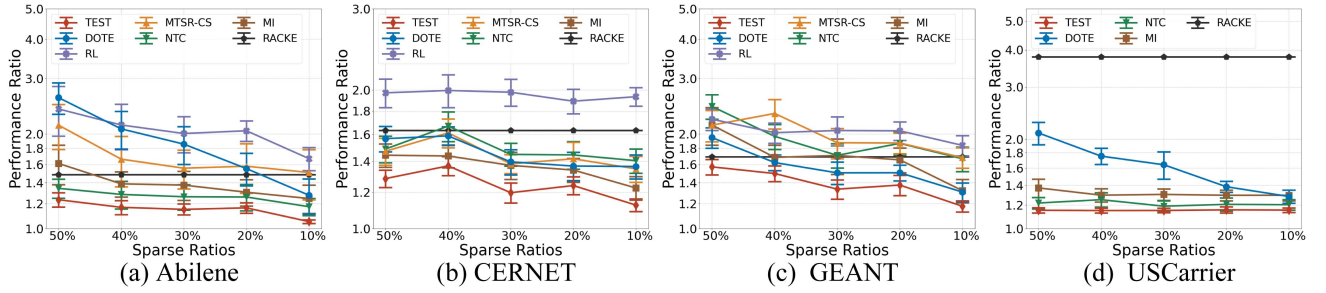


Fig. 7. TE comparison under node-level measurement.

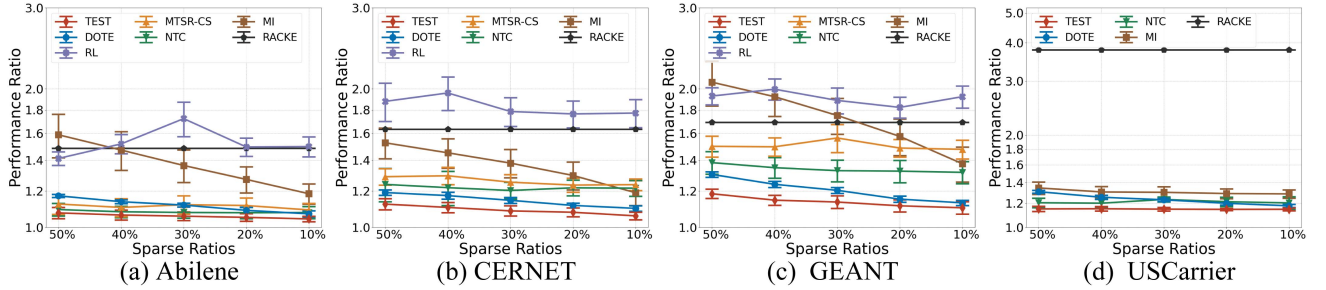


Fig. 8. TE comparison under flow-level measurement.

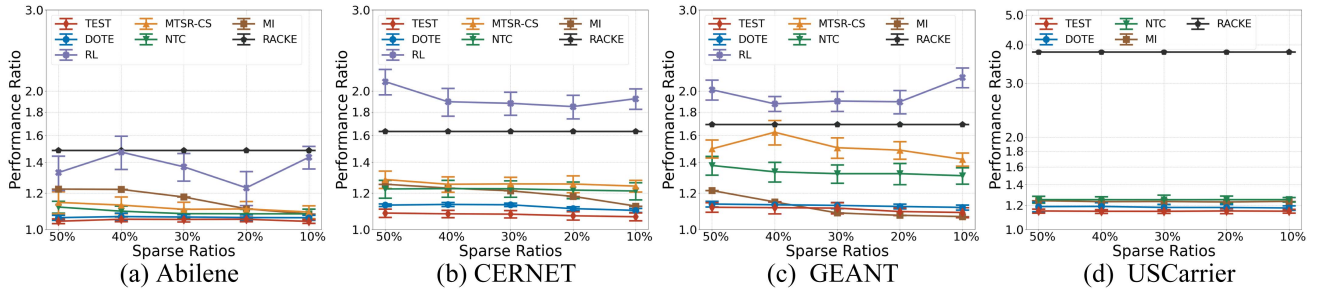


Fig. 9. TE comparison under optimal top-k flow-level measurement.

maximum flow prediction in each routing cycle, and then obtain the routing policy via LS2SR algorithm [43].

- **DOTE [12]:** It models TE optimization as a stochastic optimization problem and employs DNNs to establish an end-to-end mapping relationship from historical TMs to the future routing policies. Similar to RL-based TE method, the input TMs are sparse TMs.

Note that in scenarios with sparse measurements, complete TMs are unavailable. Thus, only sparse TMs are used during the training phase, while in the testing phase, routing policies are generated from sparse TMs and evaluated on the complete TMs. In addition, due to the high memory consumption of large-scale topologies, we did not implement the MTSR-CS and RL methods on USCarrier.

### B. TE Performance Comparison

We conduct the comparative experiments under node-level, flow-level measurement, and top-k flow-level measurement schemes across various topologies, with sparse ratios ranging from 50% to 10%. As shown in Figs. 7–10, the curves of

TEST are almost below those of the baseline methods across all topologies and sparse ratios under all measurement schemes, indicating TEST's superior performance in reducing the MLU. Concretely, at the node-level measurement scheme, compared to the baseline methods in different topologies, TEST reduces the MLU by Abilene (8.77%–43.30%), CERNET (8.85%–36.39%), GEANT (11.34%–31.79%) and USCarrier (6.67%–69.56%), respectively. At the flow-level measurement scheme, compared to the baseline methods in different topologies, TEST reduces the MLU by Abilene (2.11%–30.49%), CERNET (4.71%–40.41%), GEANT (5.65%–41.63%) and USCarrier (6.85%–69.73%), respectively. At the optimal top-k flow-level measurement scheme, compared to the baseline methods across different topologies, TEST reduces the MLU by Abilene (1.38%–29.43%), CERNET (3.94%–44.07%) and USCarrier (2.94%–69.72%), respectively. It is worth noting that MI outperforms TEST at low sparse ratios on GEANT. This occurs because, under lower sparsity, the sampled flows contain a higher proportion of large flows, which provides stronger guidance for MI to reduce MLU. Additionally, the MCF-based optimization in MI generates routing solutions that are closer to optimal under

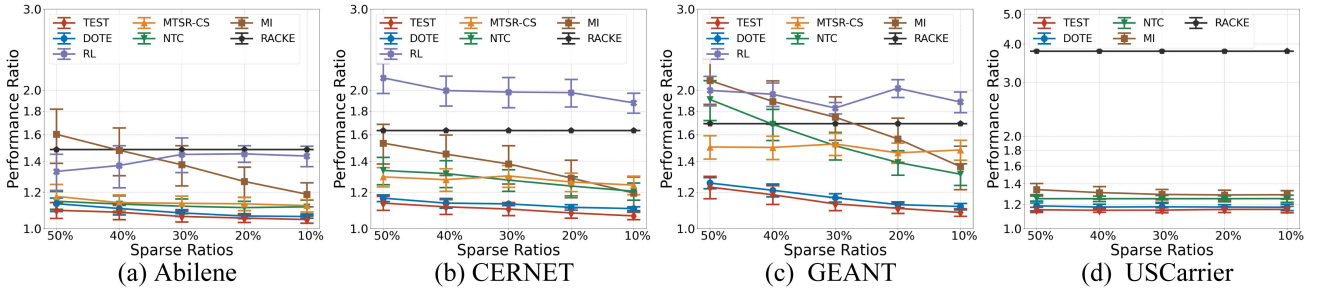


Fig. 10. TE comparison under practical top-k flow-level measurement.

these conditions. At the practical top-k flow-level measurement scheme, compared to the baseline methods in different topologies, TEST reduces the MLU by Abilene (1.86%–28.03%), CERNET (2.63%–44.68%), GEANT (2.40%–40.72%) and USCarrier (2.22%–69.61%), respectively. In such scenarios, MI exhibits degraded performance under the practical top-k strategy because the used approximate sampling may miss some of the most significant flows. In most cases, the curves of TEST decline as the sparse ratio decreases. However, in certain cases, such as the node-level scheme on Cernet, TEST performs better at a sparse ratio of 30% than at 20%. We believe the possible reason is that the sparse TM captured by random measurements may lead to different degrees of information loss. This information loss can simplify the data and, in some cases, may help reduce the complexity of establishing the mapping between sparse TMs and optimal routing policies. As shown in the USCarrier results (Figs. 7–10), the MI and NTC methods achieve performance comparable to that of TEST. This can be attributed to the fact that the traffic flows in USCarrier are generated using a gravity model rather than real-world traffic traces. As a result, the missing traffic in USCarrier can be more easily reconstructed by MI and NTC, thereby improving the quality of the routing policies derived from these methods.

TEST outperforms RACKE because RACKE’s worst-case optimization for all possible TMs limits its adaptability to real-world traffic patterns, hindering dynamic performance optimization. TEST outperforms MI because MI depends on the current network traffic patterns and the balance of demands among neighboring nodes, leading to suboptimal performance in dynamic networks. TEST outperforms RL-based TE because it is challenging for the RL agent to effectively explore and learn optimal routing policies under the sparse traffic measurements. TEST outperforms NTC because NTC heavily relies on the reconstructed and predicted TMs, where reconstruction and prediction errors may significantly impact its performance. TEST outperforms MTSR-CS because MTSR-CS depends on the accuracy of compressive sensing techniques, which may struggle to learn effectively without complete TMs. TEST demonstrates improved performance over DOTE, as DOTE is designed to optimize routing based on complete TMs. The DNN model used in DOTE struggles to effectively capture traffic patterns from incomplete TMs, resulting in suboptimal routing policies when applied to complete TMs and a noticeable degradation in TE performance, particularly under lower sparse ratios.

### C. Robustness to Link Failures and Traffic Changes

To evaluate the robustness of the proposed TEST method against link failures, we conduct experiments in scenarios where failures are simulated by randomly removing a subset of links. Figs. 11, 12 and 13 present the MLU performance comparisons under different sparsity ratios for different measurement schemes across the Abilene, CERNET, and GEANT topologies, respectively. It can be observed that TEST consistently achieves the lowest MLU performance ratio compared to all other methods under link failure scenarios.

Specifically, under node-level measurement scheme, TEST improves TE performance by Abilene (12.52%–51.83%), CERNET (2.18–26.99%) and GEANT (2.05%–30.17%) in different numbers of link failures. Under flow-level measurement scheme, TEST improves TE performance by Abilene (2.27%–17.87%), CERNET (0.48%–19.72%) and GEANT (1.56%–24.14%). Under optimal top-k measurement scheme, TEST improves TE performance by Abilene (6.98%–28.70%), CERNET (3.42%–37.31%) and GEANT (0.45%–38.28%). Under practical top-k measurement scheme, TEST improves TE performance by Abilene (2.76%–26.68%), CERNET (1.45%–36.70%) and GEANT (1.30%–36.11%). This is because the proposed TEST method is trained on synthetically generated traffic patterns with high diversity, which enhances its robustness against link failures.

To evaluate the robustness of TEST to traffic changes, we follow the approach used in [13] and simulate traffic variations by adding noise to the testing TMs. Specifically, we first generate the traffic noises by sampling from a Gaussian distribution  $\mathcal{N}(0, \sigma)$ . Then, the generated traffic noises are scaled by a factor  $\alpha$ . Finally, the generated traffic noises treated as traffic variations are added to the testing TMs. Here,  $\sigma$  denotes the standard deviation of each traffic flow. In our experiments, we set  $\alpha$  to 0.5, 0.75, 1.0. Table III report the performance degradation of the TEST routing model trained under the different measurement schemes, respectively, across different traffic changes. We observe that the performance of TEST exhibits varying degrees of degradation as the factor  $\alpha$  increases. Across all measurement schemes, the maximum performance degradation remains bounded under varying topologies and sparse ratios, specifically within 14.01% for node-level, 14.83% for flow-level, 18.21% for optimal top-k, and 16.32% for practical top-k. These results demonstrate that the proposed TEST model exhibits a certain

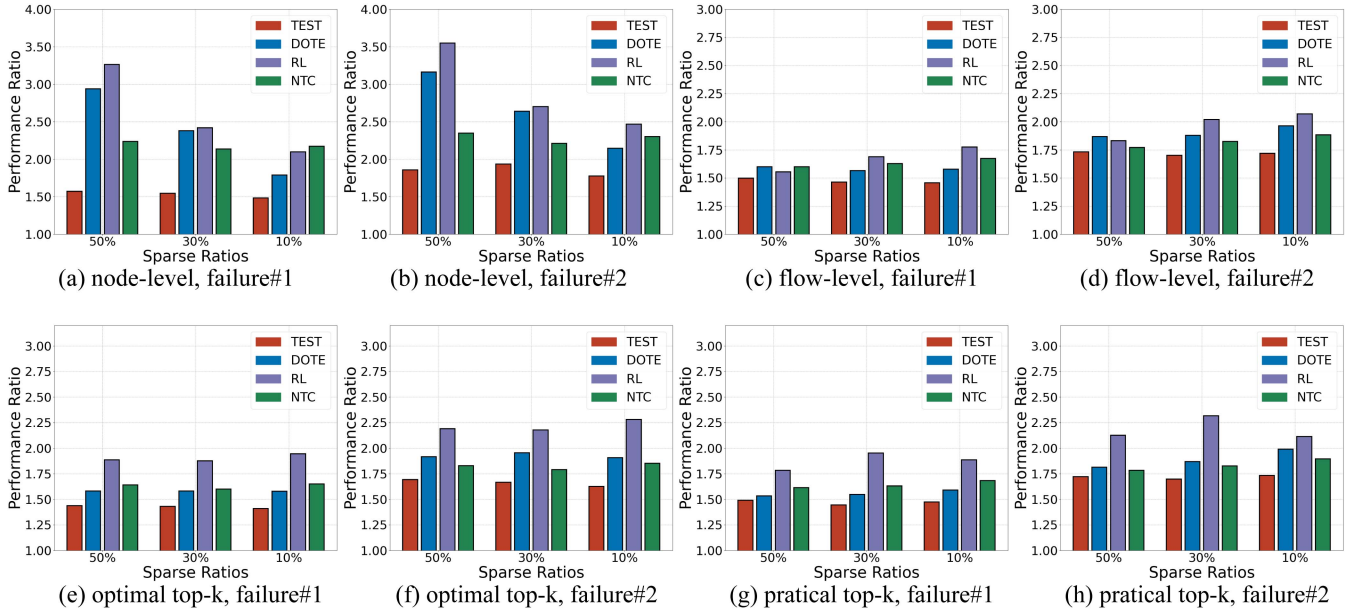


Fig. 11. MLU performance with different numbers of link failures under measurement schemes in Abilene.

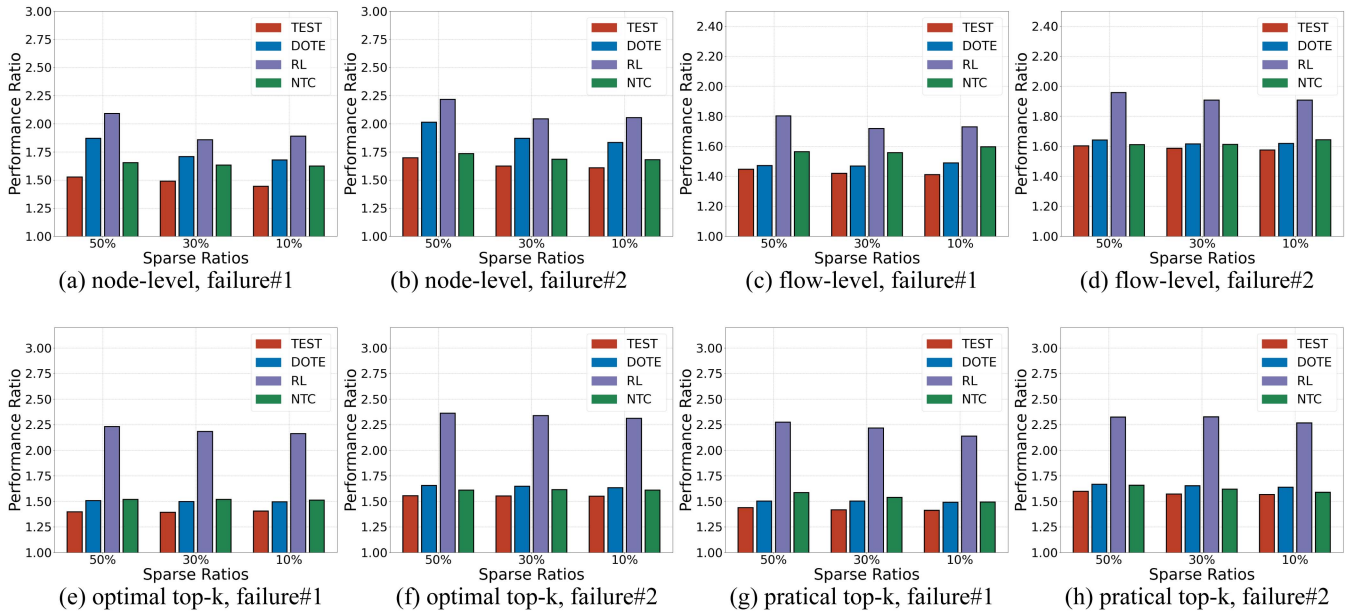


Fig. 12. MLU performance with different numbers of link failures under measurement schemes in CERNET.

degree of robustness to traffic changes, owing to its ability to generate routing policies that adapt to diverse traffic patterns.

#### D. Ablation Study

Here, we conduct ablation studies by replacing or removing key components in TEST under both measurement schemes, with sparse ratios ranging from 50% to 10% across different topologies. Specifically, for the Transformer module, we replace it with an LSTM (RepLSTM) and also examine the impact of its removal (WoTFM). For the GCN module, we substitute it

with a GAT (RepGAT) and also assess the effect of its removal (WoGCN). For the Temporal Attention module, we evaluate its removal by using only the output of the Transformer (WoTA). As shown in Figs. 14 and 15, under the node-level measurement scheme, TEST achieves average MLU performance improvements over RepLSTM (2.64%), RepGAT (2.71%), WoTA (2.43%), WoTFM (2.87%), and WoGCN (3.21%) across various topologies. Under the flow-level measurement scheme, the improvements are 0.63% for RepLSTM, 0.82% for RepGAT, 0.83% for WoTA, 0.60% for WoTFM, and 0.75% for WoGCN. These results indicate that the components designed



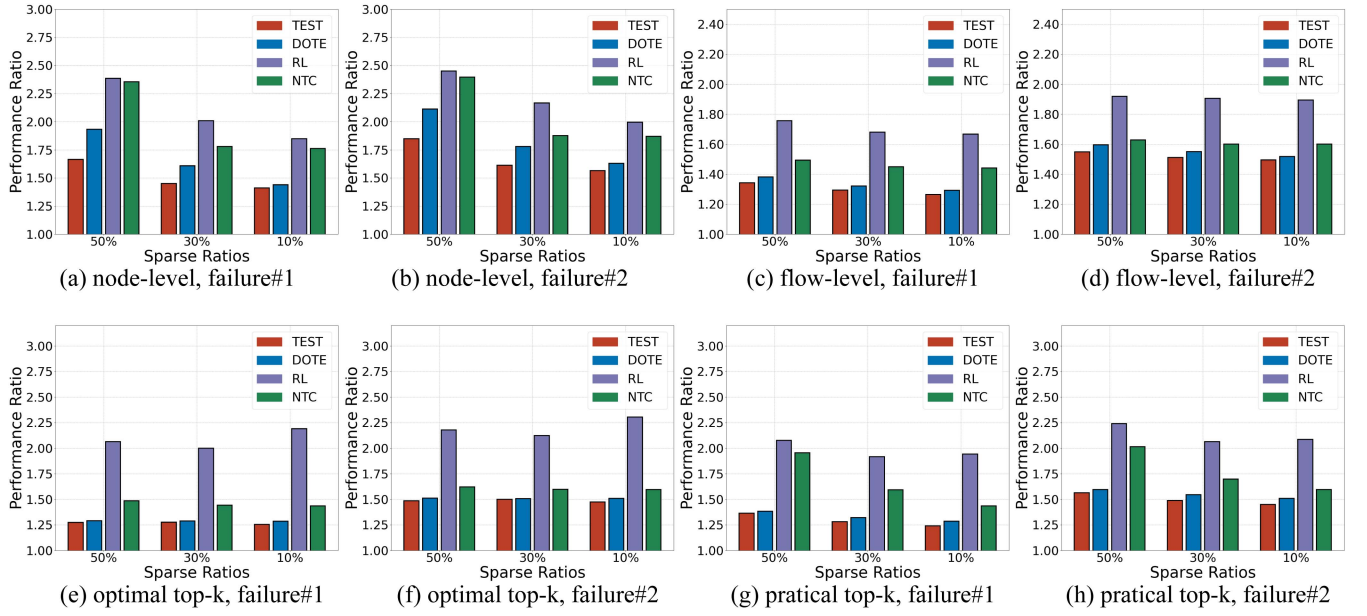


Fig. 13. MLU performance with different numbers of link failures under measurement schemes in GEANT.

TABLE III  
PERFORMANCE DEGRADATION OF TEST WITH TRAFFIC CHANGES

| Measurement Schemes |              | Node            |       |        | Flow            |       |        | Optimal Top-k   |        |        | Practical Top-k |        |        |
|---------------------|--------------|-----------------|-------|--------|-----------------|-------|--------|-----------------|--------|--------|-----------------|--------|--------|
| Topology            | Sparse Ratio | Factor $\alpha$ |       |        | Factor $\alpha$ |       |        | Factor $\alpha$ |        |        | Factor $\alpha$ |        |        |
|                     |              | 0.5             | 0.75  | 1.0    | 0.5             | 0.75  | 1.0    | 0.5             | 0.75   | 1.0    | 0.5             | 0.75   | 1.0    |
| Abilene             | 50%          | 0.69%           | 1.32% | 2.25%  | 0.69%           | 1.32% | 2.25%  | 0.95%           | 2.27%  | 3.64%  | 0.70%           | 1.43%  | 2.65%  |
|                     | 30%          | 0.82%           | 1.74% | 2.93%  | 0.82%           | 1.74% | 2.93%  | 0.77%           | 1.86%  | 3.54%  | 0.66%           | 1.64%  | 2.85%  |
|                     | 10%          | 1.01%           | 1.10% | 3.00%  | 1.01%           | 1.10% | 3.00%  | 1.28%           | 2.27%  | 3.68%  | 0.89%           | 2.02%  | 3.45%  |
| CERNET              | 50%          | 3.30%           | 6.63% | 11.59% | 3.70%           | 6.63% | 11.59% | 5.58%           | 10.88% | 16.57% | 3.39%           | 7.57%  | 12.60% |
|                     | 30%          | 4.01%           | 8.51% | 13.30% | 4.01%           | 8.51% | 13.30% | 5.43%           | 10.84% | 18.04% | 3.97%           | 9.19%  | 14.15% |
|                     | 10%          | 4.30%           | 9.14% | 14.83% | 4.30%           | 9.14% | 14.83% | 5.90%           | 11.61% | 18.21% | 4.98%           | 10.43% | 16.32% |
| GEANT               | 50%          | 0.93%           | 2.11% | 3.68%  | 0.93%           | 2.11% | 3.68%  | 1.81%           | 4.15%  | 6.77%  | 0.86%           | 1.93%  | 3.54%  |
|                     | 30%          | 1.30%           | 2.73% | 4.10%  | 1.30%           | 2.73% | 4.90%  | 2.07%           | 4.40%  | 7.26%  | 1.59%           | 3.11%  | 5.56%  |
|                     | 10%          | 1.10%           | 4.17% | 6.88%  | 1.90%           | 4.17% | 6.88%  | 2.31%           | 4.76%  | 8.64%  | 2.22%           | 4.83%  | 7.66%  |
| USCarrier           | 50%          | 1.84%           | 3.83% | 7.52%  | 1.84%           | 3.83% | 7.52%  | 5.04%           | 9.36%  | 13.95% | 5.06%           | 9.16%  | 14.23% |
|                     | 30%          | 1.64%           | 3.86% | 7.61%  | 1.64%           | 3.86% | 7.61%  | 5.06%           | 9.25%  | 14.10% | 5.10%           | 9.40%  | 14.10% |
|                     | 10%          | 1.76%           | 4.41% | 7.76%  | 1.76%           | 4.41% | 7.76%  | 5.28%           | 9.10%  | 13.78% | 5.02%           | 9.03%  | 13.74% |

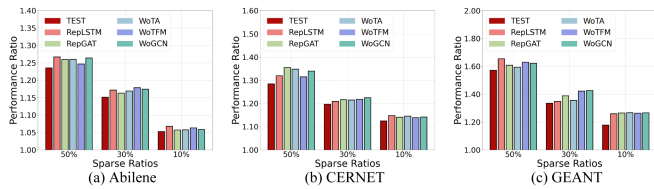


Fig. 14. Modules ablation under node-level measurement.

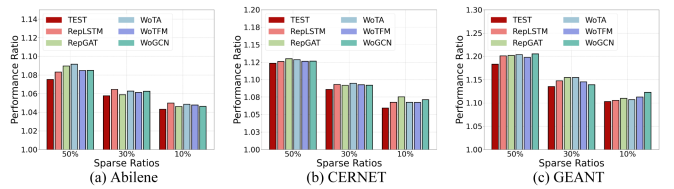


Fig. 15. Modules ablation under flow-level measurement.

in the TEST has contributed to the performance improvement. This is because the Transformer module effectively captures long-range temporal relationships in the sparse TM sequence, providing sufficient routing evidence. Additionally, the Temporal Attention module helps the Transformer module further fuse

the features extracted from different sparse TMs by evaluating their importance. Moreover, the GCN module offers advantages over traditional neural networks in modeling graph-structured data.

TABLE IV  
IMPLEMENTATION TIME FOR OFFLINE TRAINING PHASE AND ONLINE TESTING PHASE

|                                       | Topology  | TEST | DOTe | NTC                | MTSR-CS            | RACKE              | LP                 |
|---------------------------------------|-----------|------|------|--------------------|--------------------|--------------------|--------------------|
| Online Testing<br>(unit: millisecond) | Abilene   | 4.34 | 0.70 | 26.05              | 1256.50            | -                  | 24.72              |
|                                       | CERNET    | 4.47 | 0.72 | 33.48              | 1450.26            | -                  | 32.13              |
|                                       | GEANT     | 4.57 | 0.79 | 101.21             | 1872.73            | -                  | 99.82              |
|                                       | USCarrier | 6.16 | 0.99 | $1.52 \times 10^4$ | -                  | -                  | $1.52 \times 10^4$ |
| Offline Training<br>(unit: second)    | Abilene   | 330  | 304  | 442                | 2550               | 96                 | -                  |
|                                       | CERNET    | 332  | 307  | 446                | $1.14 \times 10^4$ | 116                | -                  |
|                                       | GEANT     | 336  | 316  | 480                | $2.11 \times 10^4$ | 1789               | -                  |
|                                       | USCarrier | 376  | 342  | 1112               | -                  | $8.66 \times 10^4$ | -                  |

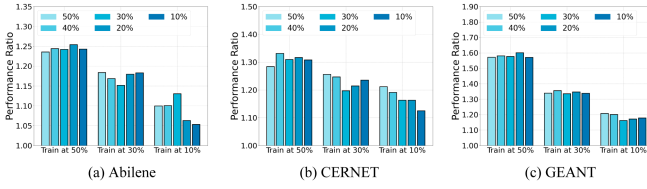


Fig. 16. Sparse generalization under node-level measurement.

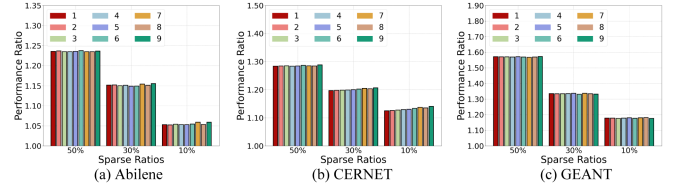


Fig. 18. Routing stability in node-level measurement.

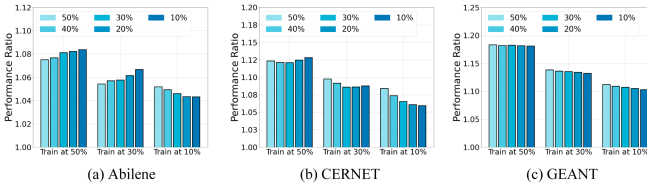


Fig. 17. Sparse generalization under flow-level measurement.

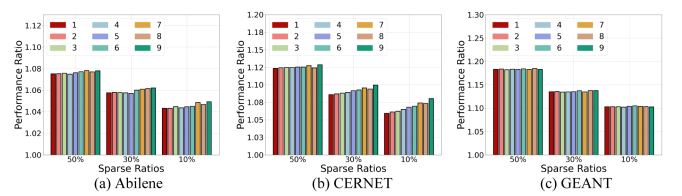


Fig. 19. Routing stability in flow-level measurement.

### E. Implementation Time

Table IV records the implementation time of various TE methods across two phases: the offline training phase and the online testing phase. During offline training, the proposed TEST method takes an average of 343.5 seconds to train a model for routing policy generation. In the online testing phase, TEST requires an average of 4.885 milliseconds to generate routing policies based on the measured sparse TMs. TEST requires less time than all methods except DOTe, primarily because it employs a neural network to directly establish the relationship between sparse TMs and routing policies, instead of solving a LP problem to compute the routing policies. TEST requires slightly more time than DOTe due to its use of a Transformer model, which has higher computational complexity compared to the convolutional neural network used in DOTe.

### F. Sparse Ratio Generalization

To further assess the generalization capability of TEST across various sparsity levels, we trained our routing model at sparse ratios of 50%, 30%, and 10%, and tested it on a range of sparse ratios from 50% to 10%. As shown in Figs. 16 and 17, the proposed TEST method trained at sparse ratios of 50%, 30%, and 10% exhibits a slight performance degradation when generalized to other sparse ratios. For node-level measurement

scheme, when TEST is trained at a sparse ratio of 50%, 30%, and 10%, the maximum degradations for Abilene, CERNET, and GEANT are (1.46%, 3.50%, 1.83%), (2.71%, 4.32%, 1.47%) and (6.85%, 7.17%, 1.47%), respectively. For flow-level measurement scheme, when TEST is trained at a sparse ratio of 50%, 30%, and 10%, the maximum degradations for Abilene, CERNET, and GEANT are (0.79%, 0.41%, -0.05%), (0.85%, 1.06%, 0.27%) and (0.81%, 2.28%, 0.81%), respectively. These results demonstrate the generalization ability of the proposed TEST method, as it maintains satisfactory network performance under different sparse ratios.

### G. Routing Stability

To evaluate the routing stability of the proposed TEST method, we apply the routing policies predicted at time  $t$  to a sequence of future TMs from  $t+1$  to  $t+9$ . As shown in Figs. 18 and 19, for both node-level and flow-level measurement schemes, the network performance gradually degrades as the measurement time increases. Specifically, under the node-level measurement scheme, the maximum performance degradation is less than 2.89%, 1.43%, and 10.22% at sparse ratios of 50%, 30%, and 10%, respectively. Under the flow-level measurement scheme, the maximum performance degradation remains below 1.29%, 1.25%, and 1.98% at sparse ratios of 50%

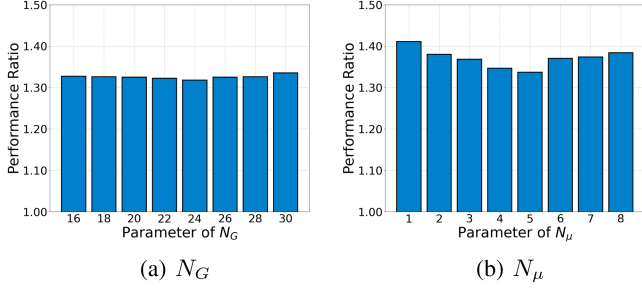


Fig. 20. Impact of the parameters used in MGM on GEANT.

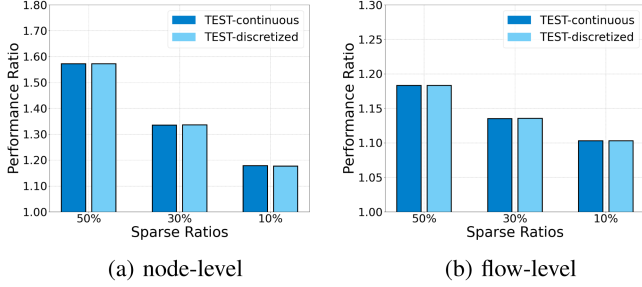


Fig. 21. The performance of TEST under continuous and discretized split ratio constraints on GEANT.

, 30% , and 10% , respectively. These results demonstrate that the routing policies generated by TEST exhibit good stability when applied to future TMs.

#### H. Parameter Sensitivity Analysis

We analyze the impact of parameters  $N_\mu$  and  $N_G$  used in Algorithm 1 on network performance in GEANT. Here,  $N_\mu$  and  $N_G$  define the number of partitions used and the number of samples generated, respectively. As shown in Fig. 20(a), the value of  $N_G$  slightly influences the MLU performance of TEST when  $N_G$  ranges from 16 to 30. Since the MLU performance reaches its peak at  $N_G = 24$ , we set  $N_G$  to 24 in our experiments. As shown in Fig. 20(b), the MLU performance gradually increases when  $N_\mu$  ranges from 1 to 5 and gradually decreases when  $N_\mu$  increases beyond 5. This is because a larger  $N_\mu$  requires considering more traffic patterns, which increases the complexity of training the model, whereas a smaller  $N_\mu$  involves fewer traffic patterns, which may reduce their diversity. Therefore, we set  $N_\mu$  to 5 in our experiments.

#### I. Routing Performance With Discretized Policy

In practice, traffic split ratios are often discretized, which limits the precision of packet flow splitting. To evaluate the impact of this constraint, we compare the TE performance of the proposed TEST method under both discretized and continuous traffic split ratio constraints. To simulate the discretized constraint, we round the continuous split ratios generated by TEST to two decimal places before deployment. Fig. 21 presents the network performance of TEST under continuous and discretized split ratio constraints on G'EAANT. We observe that converting the continuous traffic split ratios to discretized ones has almost no

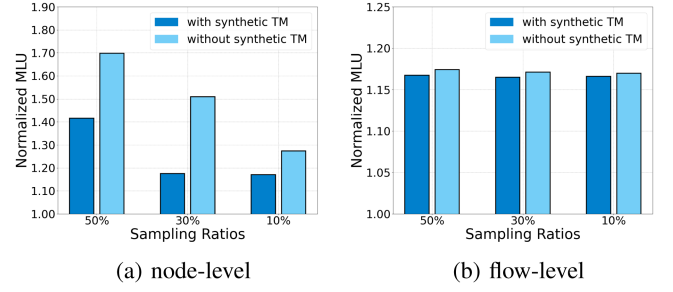


Fig. 22. The performance of TEST evaluated under the balanced and unbalanced datasets.

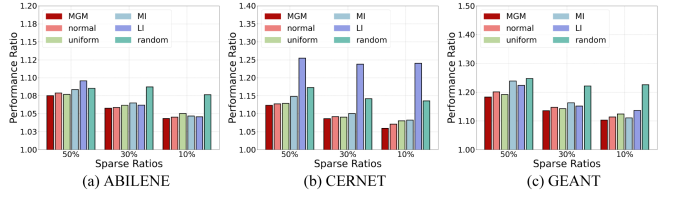


Fig. 23. The performance of TEST with various traffic synthesis schemes.

impact on performance. These results indicate that the proposed TEST method adapts well to the discretized split ratio constraint imposed by the actual network environment.

#### J. Impact of Balance and Unbalance Traffic Demands

To further evaluate the influence of unbalanced traffic patterns on TEST, we simulate two types of traffic datasets: a balanced dataset and an unbalanced dataset. Specifically, the traffic flows in both datasets are sampled from a normal distribution with the same mean traffic volume of 1000, but with different standard deviations. The standard deviation is set to 10 for the balanced dataset and 900 for the unbalanced dataset. Additionally, to evaluate the effectiveness of synthetic TMs, we compare the performance of TEST with and without using the proposed synthetic TMs on both datasets. As shown in Fig. 22, using the proposed synthetic TMs significantly improves the performance of TEST for both unbalanced and balanced traffic patterns, particularly in the node-level measurement scheme. This further demonstrates that our strategy of adaptively constructing a set of synthesized TMs with diverse traffic patterns enhances the robustness of the generated routing policies to unknown traffic demands.

#### K. Impact of Alternative Traffic Synthesis Schemes

To further evaluate the impact of alternative traffic synthesis schemes on TEST under sparse measurements, we replace MGM with other approaches, including normal, uniform, and random sampling, as well as linear interpolation (LI) and mean interpolation (MI). In each scheme, the missing traffic in the sparse TM is generated either by sampling from statistical distributions or by applying interpolation rules derived from historical measurements (e.g., constructing a normal distribution using the mean and variance of past traffic). As shown in Fig. 23, the uniform, normal, random, MI, and LI schemes all yield



lower performance than our MGM approach, demonstrating the superiority of leveraging historical traffic to recover diverse traffic patterns. Such recovered diverse traffics enable the proposed TEST method to generate more robust routing policies for unseen traffic demands.

## VII. CONCLUSION

In this paper, we have proposed an end-to-end learning-based TE method, TEST, to achieve TE under dynamic traffic scenarios with sparse traffic measurements. To improve the adaptability of the routing model to unknown traffic demands in sparse TMs, we have proposed optimizing routing under a set of synthesized diverse TMs, which are adaptively generated. To capture the spatio-temporal features among the sparse TMs, we have integrated Transformer model with GCN into the routing model to provide sufficient routing evidence. Extensive comparative experiments across various network topologies have demonstrated the superiorities of the proposed TEST in enhancing TE performance with incomplete TMs measured at different sparsity ratios. Additionally, the analysis on network scenarios involving link failures and dynamic traffic changes has further highlighted resilience and adaptability of the routing policies generated by TEST. We believe that our study presents a promising research direction for effectively implementing centralized TE while significantly reducing measurement overhead, making it a practical solution for dynamic and large-scale networks. Furthermore, owing to its low measurement overhead, our approach also offers great potential for enabling centralized TE in SDN-enabled mobile networks. Additionally, although various other sparse measurement schemes exist, a comprehensive evaluation of TEST across these methods is planned for future work.

## REFERENCES

- [1] J. Zhang, K. Xi, and H. J. Chao, "Load balancing in IP networks using generalized destination-based multipath routing," *IEEE/ACM Trans. Netw.*, vol. 23, no. 6, pp. 1959–1969, Dec. 2015.
- [2] Z. Guo, W. Chen, Y.-F. Liu, Y. Xu, and Z.-L. Zhang, "Joint switch upgrade and controller deployment in hybrid software-defined networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1012–1028, May 2019.
- [3] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proc. IEEE Conf. Comput. Commun.*, 2000, pp. 519–528.
- [4] D. Thaler and C. Hopps, "Multipath issues in unicast and Multicast next-hop selection," document IETF RFC 2991, Nov. 2000.
- [5] H. Racke, "Minimizing congestion in general networks," in *Proc. 43rd Annu. IEEE Symp. Found. Comput. Sci.*, 2002, pp. 43–52.
- [6] P. Kumar et al., "{Semi-oblivious} traffic engineering: The road not taken," in *Proc. 15th USENIX Symp. Netw. Syst. Des. Implementation*, 2018, pp. 157–170.
- [7] K. Holmberg and D. Yuan, "Optimization of Internet protocol network design and routing," *Netw.: Int. J.*, vol. 43, no. 1, pp. 39–53, 2004.
- [8] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008.
- [9] Q. He et al., "Routing optimization with deep reinforcement learning in knowledge defined networking," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1444–1455, Feb. 2024.
- [10] M. Ye, J. Zhang, Z. Guo, and H. J. Chao, "LARRI: Learning-based adaptive range routing for highly dynamic traffic in WANs," in *Proc. IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.
- [11] M. Ye, J. Zhang, Z. Guo, and H. J. Chao, "Roracle: Enabling lookahead routing for scalable traffic engineering with supervised learning," in *Proc. IEEE 31st Int. Conf. Netw. Protoc.*, 2023, pp. 1–11.
- [12] Y. Perry et al., "{DOTE}: Rethinking (predictive){ WAN} traffic engineering," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation*, 2023, pp. 1557–1581.
- [13] X. Liu, S. Zhao, and Y. Cui, "FIGRET: Fine-grained robustness-enhanced traffic engineering," 2024, *arXiv:2405.04932*.
- [14] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route," in *Proc. 16th ACM Workshop Hot Topics Netw.*, 2017, pp. 185–191.
- [15] Z. Xu et al., "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 1871–1879.
- [16] Y. Guo et al., "Traffic engineering in hybrid software defined network via reinforcement learning," *J. Netw. Comput. Appl.*, vol. 189, 2021, Art. no. 103116.
- [17] P. L. Ventre et al., "Segment Routing: A comprehensive survey of research activities, standardization efforts, and implementation results," *IEEE Commun. Surv. Tut.*, vol. 23, no. 1, pp. 182–221, First Quarter 2021.
- [18] R. Amin, E. Rojas, A. Aqdas, S. Ramzan, D. Casillas-Perez, and J. M. Arco, "A survey on machine learning techniques for routing optimization in SDN," *IEEE Access*, vol. 9, pp. 104582–104611, 2021.
- [19] Y. Fang, A. F. Diallo, C. Zhang, and P. Patras, "Spider: Deep learning-driven sparse mobile traffic measurement collection and reconstruction," in *Proc. IEEE Glob. Commun. Conf.*, 2021, pp. 1–6.
- [20] J. Zhong, H. Liu, and L. Yu, "Sparse regularization for traffic load monitoring using bridge response measurements," *Measurement*, vol. 131, pp. 173–182, 2019.
- [21] D. Jiang, W. Wang, L. Shi, and H. Song, "A compressive sensing-based approach to end-to-end network traffic reconstruction," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 1, pp. 507–519, First Quarter 2020.
- [22] K. Xie, J. Tian, G. Xie, G. Zhang, and D. Zhang, "Low cost sparse network monitoring based on block matrix completion," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
- [23] G. Jia, P. Miller, X. Hong, H. Kaluturage, and T. Ban, "Anomaly detection in network traffic using dynamic graph mining with a sparse autoencoder," in *Proc. 18th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./13th IEEE Int. Conf. Big Data Sci. Eng.*, 2019, pp. 458–465.
- [24] W. Guo, J. Li, X. Liu, and Y. Yang, "Privacy-preserving compressive sensing for real-time traffic monitoring in urban city," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14510–14522, Dec. 2020.
- [25] K. Xie et al., "Accurate recovery of missing network measurement data with localized tensor completion," *IEEE/ACM Trans. Netw.*, vol. 27, no. 6, pp. 2222–2235, Dec. 2019.
- [26] K. Xie et al., "Deep adversarial tensor completion for accurate network traffic measurement," *IEEE/ACM Trans. Netw.*, vol. 31, no. 5, pp. 2101–2116, Oct. 2023.
- [27] Z. Tang, Y. Bao, and H. Li, "Group sparsity-aware convolutional neural network for continuous missing data recovery of structural health monitoring," *Struct. Health Monit.*, vol. 20, no. 4, pp. 1738–1759, 2021.
- [28] B. Fortz and M. Thorup, "Optimizing OSPF/IS-IS weights in a changing world," *IEEE J. Sel. Areas Commun.*, vol. 20, no. 4, pp. 756–767, May 2002.
- [29] N. Wang, K. H. Ho, G. Pavlou, and M. Howarth, "An overview of routing optimization for Internet traffic engineering," *IEEE Commun. Surv. Tut.*, vol. 10, no. 1, pp. 36–56, First Quarter 2008.
- [30] S. Kandula, D. Katabi, B. Davie, and A. Charny, "Walking the tightrope: Responsive yet stable traffic engineering," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 4, pp. 253–264, 2005.
- [31] A. Elwalid, C. Jin, S. Low, and I. Widjaja, "MATE: MPLS adaptive traffic engineering," in *Proc. IEEE Conf. Comput. Commun. 20th Annu. Joint Conf. IEEE Comput. Commun. Soc.*, 2001, pp. 1300–1309.
- [32] Z. Liu, Z. Wang, X. Yin, X. Shi, Y. Guo, and Y. Tian, "Traffic matrix prediction based on deep learning for dynamic traffic engineering," in *Proc. IEEE Symp. Comput. Commun.*, 2019, pp. 1–7.
- [33] S. Troia, F. Sapienza, L. Varé, and G. Maier, "On deep reinforcement learning for traffic engineering in SD-WAN," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2198–2212, Jul. 2021.
- [34] W. Zheng, Y. Li, M. Hong, X. Fan, and G. Zhao, "Flow-by-flow traffic matrix prediction methods: Achieving accurate, adaptable, low cost results," *Comput. Commun.*, vol. 194, pp. 348–360, 2022.
- [35] M. Ding, Y. Guo, Z. Huang, B. Lin, and H. Luo, "GROM: A generalized routing optimization method with graph neural network and deep reinforcement learning," *J. Netw. Comput. Appl.*, vol. 229, 2024, Art. no. 103927.
- [36] B. Lin, Y. Guo, H. Luo, and M. Ding, "TITE: A transformer-based deep reinforcement learning approach for traffic engineering in hybrid SDN with dynamic traffic," *Future Gener. Comput. Syst.*, vol. 161, pp. 95–105, 2024.

- [37] Y. Guo, M. Ding, W. Zhou, B. Lin, C. Chen, and H. Luo, "MATE: A multi-agent reinforcement learning approach for traffic engineering in hybrid software defined networks," *J. Netw. Comput. Appl.*, vol. 231, 2024, Art. no. 103981.
- [38] K. Xie, L. Wang, X. Wang, G. Xie, J. Wen, and G. Zhang, "Accurate recovery of Internet traffic data: A tensor completion approach," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun.*, 2016, pp. 1–9.
- [39] K. Xie et al., "Neural tensor completion for accurate network monitoring," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 1688–1697.
- [40] M. Roughan, M. Thorup, and Y. Zhang, "Traffic engineering with estimated traffic matrices," in *Proc. 3rd ACM SIGCOMM Conf. Internet Meas.*, 2003, pp. 248–258.
- [41] T. T. Le et al., "GCRINT: Network traffic imputation using graph convolutional recurrent neural network," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.
- [42] V. A. Le, Y. Ji, H. H. Tran, P. Le Nguyen, and J. C. S. Lui, "Achieving multi-time-step segment routing via traffic prediction and compressive sensing techniques," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1534–1549, Apr. 2024.
- [43] T. T. Le et al., "Multi-time-step segment routing based traffic engineering leveraging traffic prediction," in *Proc. IFIP/IEEE Int. Symp. Integr. Netw. Manage.*, 2021, pp. 125–133.
- [44] B. Claise, "Cisco systems netflow services export version 9," RFC 3954, Oct. 2004. [Online]. Available: <https://www.rfc-editor.org/info/rfc3954>
- [45] A. M. TootoonchianGhobadi and Y. Ganjali, "OpenTM: Traffic matrix estimator for openflow networks," in *Proc. 11th Int. Conf. Passive Act. Meas.*, Berlin, Heidelberg, Springer, 2010, pp. 201–210.
- [46] K. Xie et al., "Recover corrupted data in sensor networks: A matrix completion solution," *IEEE Trans. Mobile Comput.*, vol. 16, no. 5, pp. 1434–1448, May 2017.
- [47] N. Garg and J. Könemann, "Faster and simpler algorithms for multicommodity flow and other fractional packing problems," *SIAM J. Comput.*, vol. 37, no. 2, pp. 630–652, 2007.
- [48] B. W. Bader et al., "MATLAB Tensor Toolbox version 2.5," Jan. 2012. [Online]. Available: <http://www.sandia.gov/tgkolda/TensorToolbox/>
- [49] Y. Liu, F. Shang, L. Jiao, J. Cheng, and H. Cheng, "Trace norm regularized CANDECOMP/PARAFAC decomposition with missing data," *IEEE Trans. Cybern.*, vol. 45, no. 11, pp. 2437–2448, 2014.
- [50] J. Y. Yen, "Finding the K shortest loopless paths in a network," *Manage. Sci.*, vol. 17, no. 11, pp. 712–716, 1971.
- [51] M. Roughan, A. Greenberg, C. Kalmanek, M. Rumsewicz, J. Yates, and Y. Zhang, "Experience in measuring backbone traffic variability: Models, metrics, measurements and meaning," in *Proc. 2nd ACM SIGCOMM Workshop Meas.*, 2002, pp. 91–92.
- [52] D. Applegate and E. Cohen, "Making intra-domain routing robust to changing and uncertain traffic demands: Understanding fundamental tradeoffs," in *Proc. Conf. Appl., technol., Architectures, Protoc. Comput. Commun.*, 2003, pp. 313–324.
- [53] progwriter, "TMgen: Traffic matrix generation tool," Accessed Jul. 18, 2024. [Online]. [Available: <https://tmgen.readthedocs.io/en/latest/>]
- [54] A. Shapiro, D. Dentcheva, and A. Ruszczyński, *Lectures on Stochastic Programming: Modeling and Theory*. Philadelphia, PA, USA: SIAM, 2021.
- [55] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [56] N. McKeown et al., "Openflow: Enabling innovation in campus networks," *SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Mar. 2008.
- [57] Y. Zhang, "Abilene," 2004. [Online]. Available: <http://userweb.cs.utexas.edu/yzhang/research/AbileneTM/>
- [58] B. Zhang, J. Bi, J. Wu, and F. Baker, "CTE: Cost-effective intra-domain traffic engineering," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 115–116, 2014.
- [59] S. Uhlig, "GÉant," 2006. [Online]. Available: <http://totem.info.ucl.ac.be/dataset.html>
- [60] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The Internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.
- [61] Y. Wang and Z. Wang, "Explicit routing algorithms for Internet traffic engineering," in *Proc. 8th Int. Conf. Comput. Commun. Netw.*, 1999, pp. 582–588.

- [62] Gurobi Optimization, LLC, "Gurobi optimizer reference manual," 2024. [Online]. Available: <https://www.gurobi.com>
- [63] H. Räcke, "Optimal hierarchical decompositions for congestion minimization in networks," in *Proc. 40th Annu. ACM Symp. Theory Comput.*, 2008, pp. 255–264.



Yingya Guo (Member, IEEE) received the BS degree in computer science from Xiamen University, China, in 2013 and the PhD degree from Tsinghua University, in 2019. During 2017–2018 and 2020–2021, she was a visiting scholar with the Tandon School of New York University and a postdoctoral researcher with the Department of Computing, Hong Kong Polytechnic University, respectively. She is currently an associate professor with the College of Computer and Data Science, Fuzhou University, Fuzhou, China. Her research interests include traffic engineering, routing optimization and software defined networking. She has authored or coauthored more than 20 research articles published in refereed journals and proceedings, including the *IEEE Transaction On Networking*, *IEEE Transactions on Network and Service Management*, *Journal of Network and Computer Applications*, *Computer Networks*, *CONEXT*, *INFOCOM* and *ICNP*.



Zebo Huang received the BS degree from the Wuhan University of Science and Technology University, in 2023, and the master's degree with the College of Computer and Data Science, Fuzhou University. His research interests include network management, traffic engineering and machine learning.



Mingjie Ding received the BS degree from the Hainan University, in 2022. He is currently working toward the MS degree with the College of Computer and Data Science, Fuzhou University. His research interests include graph neural network, traffic engineering and reinforcement learning.



Huan Luo received the PhD degree in computer science and technology from Xiamen University, Xiamen, China, in 2017. He is currently an associate professor with the Faculty of College of Computer and Data Science, Fuzhou University, Fuzhou, China. He has authored or coauthored more than 30 research articles published in refereed journals and proceedings, including the *Journal of Network and Computer Applications*, the *Computer Networks*, the *IEEE Transactions On Geoscience And Remote Sensing*, and the *IEEE Transactions on Intelligent Transportation Systems*. His research interests include traffic engineering, computer networking, computer vision, and machine learning.